DONIFY

Plateforme de Gestion des Dons de Sang

DOCUMENTATION TECHNIQUE

Membres :
Karima Haddad
Mayssa Chikh Ali
Nadine Ben Makhlouf
Nour Khelifi
-IGL3-

1. Présentation du Projet

Nom du projet : Donify
Type : Plateforme intelligente de gestion des dons de sang
Objectif : Optimiser la mise en relation entre hôpitaux et donneurs, tout en anticipant les pénuries grâce à l'intelligence artificielle.

1.1 Objectifs Clés

- Améliorer la réactivité des établissements de santé
- Centraliser le suivi des demandes, notifications et validations
- Identifier les donneurs les plus compatibles grâce à un algorithme de matching Top-K
- Prédire les risques de pénurie de sang via un modèle IA dédié

1.2 Fonctionnalités Principales

- Authentification des utilisateurs selon leur rôle (hôpital / donneur)
- Gestion des profils hôpitaux et donneurs
- Création et consultation des demandes de sang
- Suivi des dons validés
- Suivi des notifications et validations associées aux demandes
- Sélection de donneurs compatibles par module IA
- Génération de prédictions de pénurie sanguine
- Tableau de bord dédié aux hôpitaux

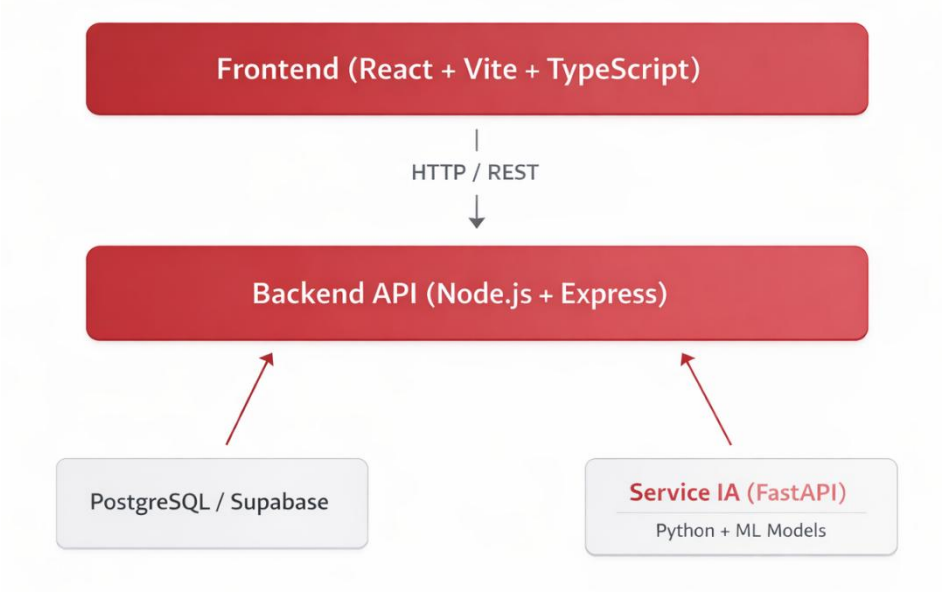
1.3 Lien GitHub

Le code source du projet est disponible sur :

2. Architecture Générale

La plateforme repose sur une architecture modulaire à trois couches, permettant une séparation claire des responsabilités et une scalabilité optimale.

2.1 Vue d'Ensemble



2.2 Description des Composants

Frontend

Le frontend fournit l'interface utilisateur de la plateforme. Il gère l'authentification, les tableaux de bord, la consultation des demandes de sang, les notifications et les profils utilisateurs. Développé avec React et TypeScript pour une robustesse maximale.

Backend API

Le backend expose les routes REST du projet. Il centralise la logique métier, l'authentification JWT, la communication avec la base de données PostgreSQL via Supabase, et orchestre les appels vers le service IA.

Service IA

Le service IA exécute les modèles de prédiction et de matching. Il expose deux fonctionnalités principales :

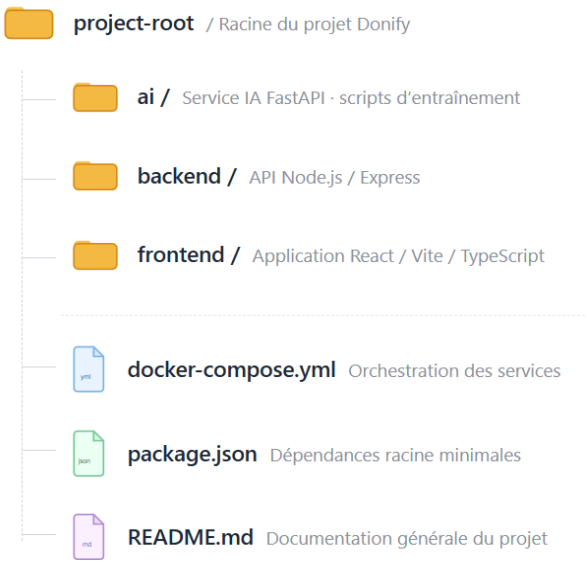
- Prédiction du niveau de pénurie sanguine (modèle XGBoost)
- Classement des donneurs compatibles selon un mécanisme Top-K

Base de Données

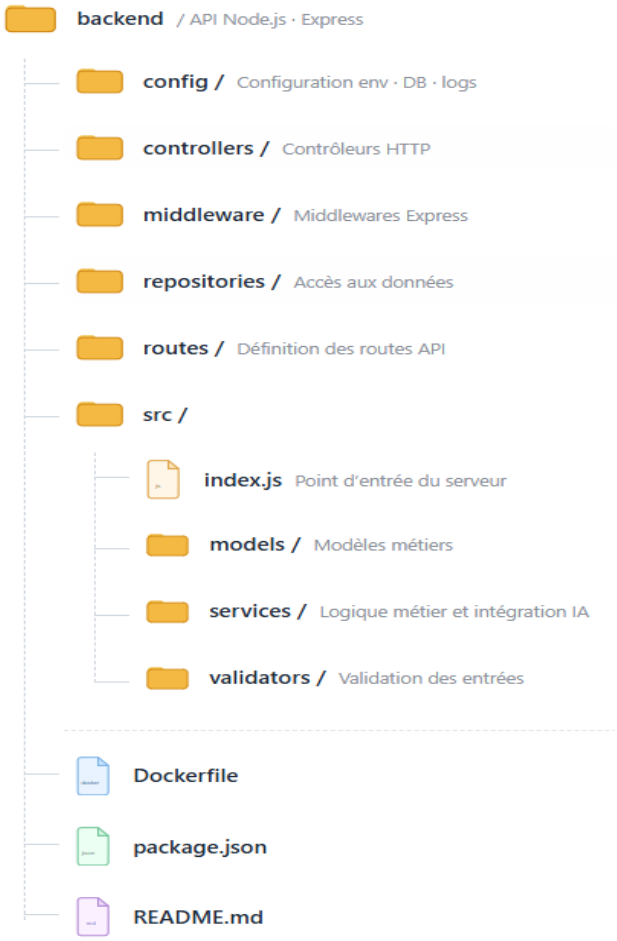
Il s'agit d'une base de données PostgreSQL hébergée en ligne via Supabase, qui centralise l'ensemble des données métier : utilisateurs, donneurs, hôpitaux, demandes de sang, donations, notifications et données de suivi opérationnel.

3. Organisation du Code

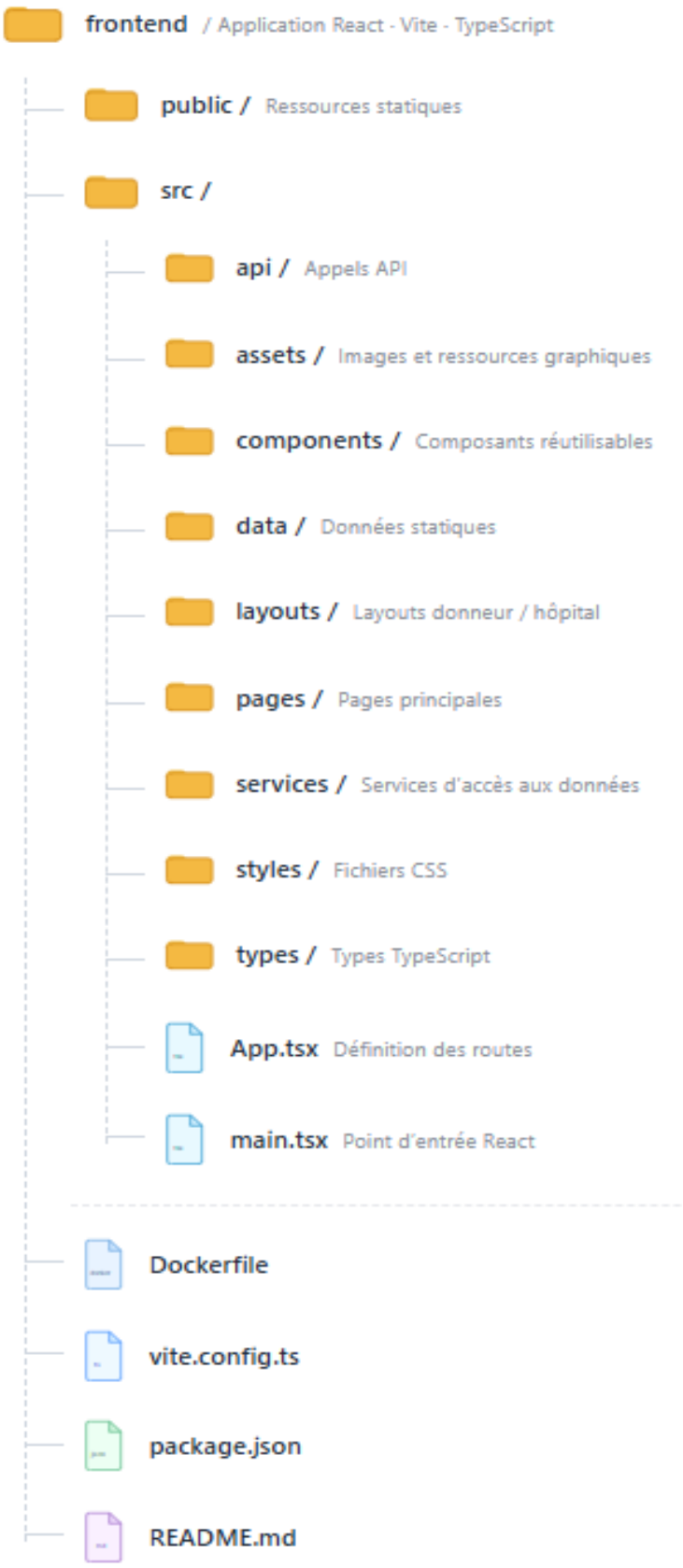
3.1 Structure du Dépôt



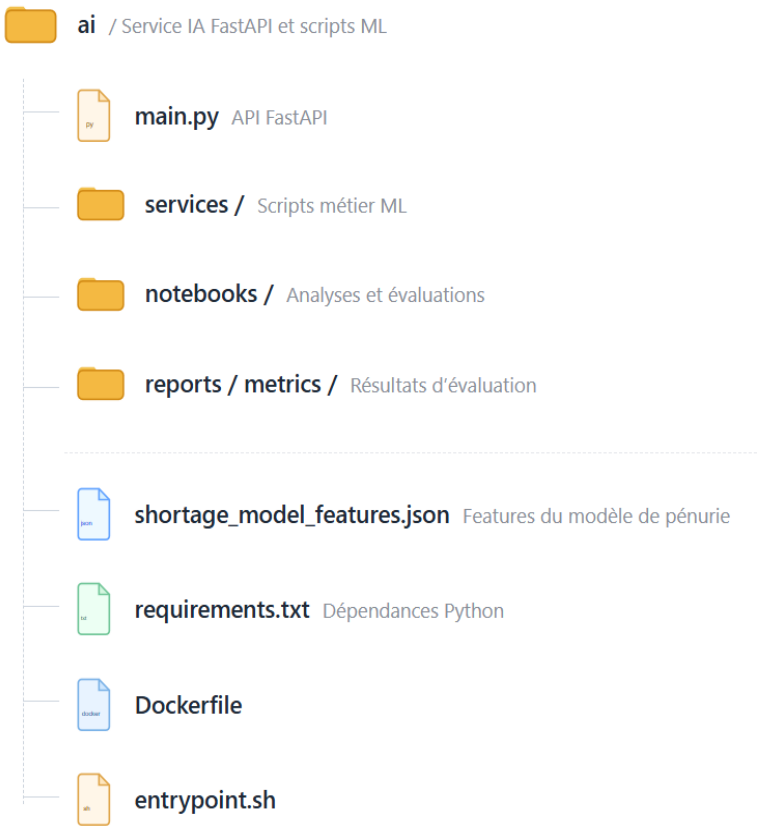
3.2 Structure du Backend



3.3 Structure du Frontend



3.4 Structure du Service IA



4. Technologies Utilisées

La stack technologique de Donify a été sélectionnée pour sa robustesse, sa maturité et sa capacité à supporter des fonctionnalités d'intelligence artificielle à l'échelle.

Couche	Technologies principales
Frontend	React, Vite, TypeScript, React Router, Axios, react-dom, react-router-dom, axios, lucide-react, react-icons, sweetalert2, react-toastify
Backend	Node.js, Express, PostgreSQL, Supabase, JWT, Zod, nodemailer, uuid, pino, pino-http, pino-pretty, cors, jsonwebtoken, bcrypt, bcryptjs, dotenv, pg
IA	Python, FastAPI, Pandas, Scikit-learn, XGBoost, Joblib, uvicorn Pydantic, dotenv-dotenv, scipy, matplotlib, seaborn
DevOps	Docker, Docker Compose
Outillage	ESLint, Nodemon

5. Prérequis

Avant d'exécuter le projet, les outils suivants doivent être installés sur la machine de développement :

- Node.js version 16 ou supérieure, avec npm
- Python version 3.10 ou supérieure, avec pip
- Docker Desktop et Docker Compose (pour l'exécution conteneurisée)

6. Variables d'Environnement

Le projet nécessite des fichiers .env pour chacun des trois services. Ces fichiers ne doivent jamais être versionnés (ils sont exclus via .gitignore).

6.1 Backend — backend/.env

PORT	4000
DATABASE_URL	ostgresql://postgres.bvtzgwqjwttuwhceyqba:Donify-2026@aws-1-eu-west-2.pooler.supabase.com:5432/postgres
AI_SERVICE_URL	http://ai:8000 (localhost si pas du Docker)
INTERNAL_API_KEY	donify_ML_Key_2026
JWT_SECRET	donify_secret_key_2026
FRONTEND_URL	http://localhost:5173
EMAIL_USER	donify.app@gmail.com
EMAIL_PASS	yzwp vvof cxxn hwgz

6.2 Frontend — frontend/.env

VITE_API_URL	http://localhost:4000

6.3 Service IA — ai/.env

INTERNAL_API_KEY	donify_ML_Key_2026

7. Préparation initiale du service IA

Avant de démarrer le serveur FastAPI pour la première fois, lancez le script de préparation :

```
cd ai
python services/entrypoint.py
```

Ce script crée les datasets et entraîne les modèles manquants si nécessaire.

8. Exécution avec Docker Compose

Le fichier docker-compose.yml à la racine du projet permet de démarrer les trois services simultanément dans des conteneurs isolés.

8.1 Cloner le Projet

```
git clone https://github.com/Karima-Haddad/Donify
cd project-root
```

8.2 Démarrage

```
# À la racine du projet
docker compose up --build
```

8.3 Services Exposés

Service	URL d'accès
Frontend	http://localhost:5173
Backend	http://localhost:4000

Service	URL d'accès
Service IA	`http://localhost:8000`

8.4 Arrêt

```
docker compose down
```

9. Installation et Exécution en Local

9.1 Lancer le Backend

```
cd backend
npm install
npm run dev
```

Le backend est disponible sur http://localhost:4000

Point de contrôle de santé : GET http://localhost:4000/health

9.2 Lancer le Frontend

```
cd frontend
npm install
npm run dev
```

Le frontend est disponible sur http://localhost:5173

9.3 Lancer le Service IA

```
cd ai
python -m venv venv
venv\Scripts\activate        # Windows
# source venv/bin/activate   # macOS / Linux
pip install -r requirements.txt
python -m uvicorn main:app --reload
```

Le service IA est disponible sur http://localhost:8000

Documentation interactive FastAPI : http://localhost:8000/docs

10. Démonstration en ligne

Accéder à la démonstration